

1. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N= 8, a[] = {5,7,3,4,9,12,6,2}

Output : Min = 2, Max = 12

Test Cases :

Input : N= 9, a[] = {1,3,5,7,9,11,13,15,17}

Output : Min = 1, Max = 17

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Aim

To find the minimum and maximum elements in an array using Divide and Conquer.

Algorithm

1. Divide array into two halves.
2. Find min and max of each half recursively.
3. Compare results and return final min and max.

Python Code

```
File Edit Format Run Options Windows Help
def minmax(a,l,h):
    if l==h:
        return a[l],a[l]

    if h==l+1:
        return min(a[l],a[h]),max(a[l],a[h])

    m=(l+h)//2

    min1,max1=minmax(a,l,m)
    min2,max2=minmax(a,m+1,h)

    return min(min1,min2),max(max1,max2)

a=[5,7,3,4,9,12,6,2]

mn,mx=minmax(a,0,len(a)-1)

print("Min =",mn)
print("Max =",mx)
```

Input

[5,7,3,4,9,12,6,2]

Output

```
>>> ===== RESTART =====
>>>
('Min =', 2)
('Max =', 12)
>>>
```

Result

The program finds minimum and maximum efficiently using Divide and Conquer.
Time complexity is $O(n)$.

2. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a Program to find both the maximum and minimum values in the array. Implement using any programming language of your choice. Execute your code and provide the maximum and minimum values found.

Input : N=8, 2,4,6,8,10,12,14,18.

Output : Min = 2, Max =18

Test Cases :

Input : N= 9, a[] = {11,13,15,17,19,21,23,35,37}

Output : Min = 11, Max = 37

Test Cases :

Input : N= 10, a[] = {22,34,35,36,43,67, 12,13,15,17}

Output : Min 12, Max 67

Aim

To find the minimum and maximum values in a sorted array.

Algorithm

1. Minimum element is first element.
2. Maximum element is last element.
3. Print both values.

Python Code

```
File Edit Format Run Options Windows Help
a=[2,4,6,8,10,12,14,18]

mn=a[0]
mx=a[0]

for i in a:
    if i<mn:
        mn=i
    if i>mx:
        mx=i

print("Min =",mn)
print("Max =",mx)
```

Input

[2,4,6,8,10,12,14,18]

Output

```
>>> ===== RESTART =====  
>>>  
('Min =', 2)  
('Max =', 18)  
>>> |
```

Result

The program directly accesses first and last elements.

Time complexity is $O(1)$.

3. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort and implement using any programming language of your choice.

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Aim

To sort an array using Merge Sort technique.

Algorithm

1. Divide array into two halves.
2. Recursively sort both halves.
3. Merge sorted halves.
4. Repeat until array is sorted.

Python Code

```
File Edit Format Run Options Windows Help
def merge_sort(a):
    if len(a)>1:
        m=len(a)//2
        l=a[:m]
        r=a[m:]

        merge_sort(l)
        merge_sort(r)

        i=j=k=0

        while i<len(l) and j<len(r):
            if l[i]<r[j]:
                a[k]=l[i]
                i+=1
            else:
                a[k]=r[j]
                j+=1
            k+=1

        while i<len(l):
            a[k]=l[i]
            i+=1
            k+=1

        while j<len(r):
            a[k]=r[j]
            j+=1
            k+=1

a=[31,23,35,27,11,21,15,28]

merge_sort(a)

print(a)
```

Input

[31,23,35,27,11,21,15,28]

Output

```
>>> ===== RESTART =====
>>>
[11, 15, 21, 23, 27, 28, 31, 35]
... |
```

Result

The program sorts the array using Merge Sort.
Time complexity is $O(n \log n)$.

4. Implement the Merge Sort algorithm in a programming language of your choice and test it on the array 12,4,78,23,45,67,89,1. Modify your implementation to count the number of comparisons made during the sorting process. Print this count along with the sorted array.

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Aim

To sort an array using Merge Sort and count the number of comparisons.

Algorithm

1. Divide array into halves recursively.
2. Merge sorted halves.
3. Count comparisons during merging.
4. Print sorted array and comparison count.

Python Code

File Edit Format Run Options Windows Help

```
count=0
```

```
def merge_sort(a):
    global count

    if len(a)>1:
        m=len(a)//2
        l=a[:m]
        r=a[m:]

        merge_sort(l)
        merge_sort(r)

        i=j=k=0

        while i<len(l) and j<len(r):
            count+=1

            if l[i]<r[j]:
                a[k]=l[i]
                i+=1
            else:
                a[k]=r[j]
                j+=1
            k+=1

        while i<len(l):
            a[k]=l[i]
            i+=1
            k+=1

        while j<len(r):
            a[k]=r[j]
            j+=1
            k+=1

a=[12,4,78,23,45,67,89,1]

merge_sort(a)

print("Sorted Array:",a)
print("Comparisons:",count)
```

Input

[12,4,78,23,45,67,89,1]

Output

```
>>> ===== RESTART =====  
>>>  
( 'Sorted Array:', [1, 4, 12, 23, 45, 67, 78, 89])  
( 'Comparisons:', 16)
```

Result

The program sorts the array using Merge Sort and counts comparisons.
Time complexity is $O(n \log n)$.

5. Given an unsorted array 10,16,8,12,15,6,3,9,5 Write a program to perform Quick Sort. Choose the first element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted.

Input : N= 9, a[] = {10,16,8,12,15,6,3,9,5}

Output : 3,5,6,8,9,10,12,15,16

Test Cases :

Input : N= 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Test Cases :

Input : N= 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82,

Aim

To sort an array using Quick Sort by choosing the first element as pivot.

Algorithm

1. Select first element as pivot.
2. Partition array into smaller and larger elements.
3. Recursively sort left and right subarrays.
4. Print sorted array.

Python Code

```
File Edit Format Run Options Windows Help
def quick_sort(a):
    if len(a)<=1:
        return a

    pivot=a[0]

    left=[x for x in a[1:] if x<=pivot]
    right=[x for x in a[1:] if x>pivot]

    return quick_sort(left)+[pivot]+quick_sort(right)

a=[10,16,8,12,15,6,3,9,5]

print(quick_sort(a))
```

Input

[10,16,8,12,15,6,3,9,5]

Output

```
>>> ===== RESTART =====
>>>
[3, 5, 6, 8, 9, 10, 12, 15, 16]
```

Result

The program sorts the array using Quick Sort recursively.
Time complexity is $O(n \log n)$ on average.

6. Implement the Quick Sort algorithm in a programming language of your choice and test it on the array 19,72,35,46,58,91,22,31. Choose the middle element as the pivot and partition the array accordingly. Show the array after this partition. Recursively apply Quick Sort on the sub-arrays formed. Display the array after each recursive call until the entire array is sorted. Execute your code and show the sorted array.

Input : N= 8, a[] = {19,72,35,46,58,91,22,31}

Output : 19,22,31,35,46,58,72,91

Test Cases :

Input : N= 8, a[] = {31,23,35,27,11,21,15,28}

Output : 11,15,21,23,27,28,31,35

Test Cases :

Input : N= 10, a[] = {22,34,25,36,43,67, 52,13,65,17}

Output : 13,17,22,25,34,36,43,52,65,67

Aim

To sort an array using Quick Sort by choosing the middle element as pivot.

Algorithm

1. Select the middle element as pivot.
2. Divide array into left, middle, and right parts.
3. Recursively apply Quick Sort on left and right subarrays.
4. Combine all parts to get sorted array.

Python Code

```
def quicksort(a):  
  
    if len(a)<=1:  
        return a  
  
    pivot=a[len(a)//2]  
  
    left=[x for x in a if x<pivot]  
    mid=[x for x in a if x==pivot]  
    right=[x for x in a if x>pivot]  
  
    print("Partition:",left,mid,right)  
  
    return quicksort(left)+mid+quicksort(right)  
  
a=[19,72,35,46,58,91,22,31]  
  
print("Sorted Array:",quicksort(a))
```

Input

[19,72,35,46,58,91,22,31]

Output

```
>>> ===== RESTART =====  
>>>  
(('Partition:', [19, 35, 46, 22, 31], [58], [72, 91])  
(('Partition:', [19, 35, 22, 31], [46], [])  
(('Partition:', [19], [22], [35, 31])  
(('Partition:', [], [31], [35])  
(('Partition:', [72], [91], [])  
(('Sorted Array:', [19, 22, 31, 35, 46, 58, 72, 91])
```

Result

The program sorts the array using Quick Sort with middle pivot. Average time complexity is $O(n \log n)$.

7. Implement the Binary Search algorithm in a programming language of your choice and test it on the array 5,10,15,20,25,30,35,40,45 to find the position of the element 20. Execute your code and provide the index of the element 20. Modify your implementation to count the number of comparisons made during the search process. Print this count along with the result.

Input : N= 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20

Output : 4

Test cases

Input : N= 6, a[] = {10,20,30,40,50,60}, search key = 50

Output : 5

Input : N= 7, a[] = {21,32,40,54,65,76,87}, search key = 32

Output : 2

Aim

To search an element using Binary Search and count comparisons.

Algorithm

1. Find middle element.
2. Compare with search key.
3. If found → return position.
4. Else search left or right half.
5. Count comparisons.

Python Code

```
File Edit Format Run Options Windows Help
a=[5,10,15,20,25,30,35,40,45]

key=20

l=0
h=len(a)-1
count=0

while l<=h:
    count+=1

    m=(l+h)//2

    if a[m]==key:
        print("Position:",m+1)
        print("Comparisons:",count)
        break

    elif a[m]<key:
        l=m+1

    else:
        h=m-1
```

Input

[5,10,15,20,25,30,35,40,45], key = 20

Output

```
>>> ===== RESTART =====
>>>
('Position:', 4)
('Comparisons:', 4)
>>> |
```

Result

The program finds the element efficiently using Binary Search.
Time complexity is $O(\log n)$.

8. You are given a sorted array 3,9,14,19,25,31,42,47,53 and asked to find the position of the element 31 using Binary Search. Show the mid-point calculations and the steps involved in finding the element. Display, what would happen if the array was not sorted, how would this impact the performance and correctness of the Binary Search algorithm?

Input : N= 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31

Output : 6

Test cases

Input : N= 7, a[] = {13,19,24,29,35,41,42}, search key = 42

Output : 7

Test cases

Input : N= 6, a[] = {20,40,60,80,100,120}, search key = 60

Output : 3

Aim

To find the position of an element in a sorted array using Binary Search.

Algorithm

1. Find middle element.
2. Compare with key.
3. If equal → return position.
4. If smaller → search right half.
5. If larger → search left half.

Python Code

```
File Edit Format Run Options Windows Help
a=[3,9,14,19,25,31,42,47,53]

key=31

l=0
h=len(a)-1

while l<=h:

    m=(l+h)//2

    print("Mid Index:",m,"Value:",a[m])

    if a[m]==key:
        print("Position:",m+1)
        break

    elif a[m]<key:
        l=m+1

    else:
        h=m-1
```

Input

[3,9,14,19,25,31,42,47,53], key = 31

Output

```
>>> ===== RESTART =====
>>>
('Mid Index:', 4, 'Value:', 25)
('Mid Index:', 6, 'Value:', 42)
('Mid Index:', 5, 'Value:', 31)
('Position:', 6)
```

Result

The program finds the element using Binary Search.

Binary Search works correctly only on sorted arrays with time complexity $O(\log n)$.

9. Given an array of points where $\text{points}[i] = [x_i, y_i]$ represents a point on the X-Y plane and an integer k , return the k closest points to the origin $(0, 0)$.

Input : $\text{points} = [[1,3],[-2,2],[5,8],[0,1]], k=2$

Output: $[[-2, 2], [0, 1]]$

(ii) Input: $\text{points} = [[1, 3], [-2, 2]], k = 1$

Output: $[[-2, 2]]$

(iii) Input: $\text{points} = [[3, 3], [5, -1], [-2, 4]], k = 2$

Output: $[[3, 3], [-2, 4]]$

Aim

To find the k closest points to the origin using distance formula.

Algorithm

1. Calculate distance of each point from origin.
2. Sort points based on distance.
3. Return first k points.

Python Code

```
File Edit Format Run Options Windows Help
points=[[1,3],[-2,2],[5,8],[0,1]]

k=2

points.sort(key=lambda p:p[0]**2+p[1]**2)

print(points[:k])
|
```


Input

points = [[1,3],[-2,2],[5,8],[0,1]], k = 2

Output

```
---  
>>> ===== RESTART =====  
>>>  
[[0, 1], [-2, 2]]  
>>> |
```

Result

The program sorts points based on distance from origin.

Time complexity is $O(n \log n)$.

10. Given four lists A, B, C, D of integer values, Write a program to compute how many tuples $n(i, j, k, l)$ there are such that $A[i] + B[j] + C[k] + D[l]$ is zero.

Input: A = [1, 2], B = [-2, -1], C = [-1, 2], D = [0, 2]

Output: 2

(ii) Input: A = [0], B = [0], C = [0], D = [0]

Output: 1

Aim

To count the number of tuples such that $A[i] + B[j] + C[k] + D[l] = 0$.

Algorithm

1. Store sums of A and B in a dictionary.
2. For each sum of C and D, check if negative exists.
3. Count matching tuples.

Python Code

```
File Edit Format Run Options Windows Help
A=[1,2]
B=[-2,-1]
C=[-1,2]
D=[0,2]

d={}
count=0

for i in A:
    for j in B:
        s=i+j
        d[s]=d.get(s,0)+1

for k in C:
    for l in D:
        s=-(k+l)
        count+=d.get(s,0)

print(count)
```

Input

```
A = [1,2]
B = [-2,-1]
C = [-1,2]
D = [0,2]
```

Output

```
>>> ===== RESTART =====
>>>
2
```

Result

The program counts tuples whose sum equals zero efficiently.
Time complexity is $O(n^2)$.

11. To Implement the Median of Medians algorithm ensures that you handle the worst-case time complexity efficiently while finding the k-th smallest element in an unsorted array.

arr = [12, 3, 5, 7, 19] k = 2 Expected Output:5

arr = [12, 3, 5, 7, 4, 19, 26] k = 3 Expected Output:5

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6 Expected Output:6

Aim

To find the k-th smallest element in an unsorted array using Divide and Conquer.

Algorithm

1. Divide array into groups of 5.
2. Find median of each group.
3. Recursively find median of medians as pivot.
4. Partition array into smaller and larger elements.
5. Recursively search required part.

Python Code

```
File Edit Format Run Options Windows Help
def kth_smallest(arr, k):
    if len(arr) <= 5:
        return sorted(arr)[k-1]

    groups = [arr[i:i+5] for i in range(0, len(arr), 5)]
    medians = [sorted(g)[len(g)//2] for g in groups]
    pivot = kth_smallest(medians, len(medians)//2+1)

    low = [x for x in arr if x < pivot]
    high = [x for x in arr if x > pivot]
    equal = [x for x in arr if x == pivot]

    if k <= len(low):
        return kth_smallest(low, k)

    elif k <= len(low) + len(equal):
        return pivot

    else:
        return kth_smallest(high, k - len(low) - len(equal))

arr = [12, 3, 5, 7, 19]

print(kth_smallest(arr, 2))
```

Input

arr = [12,3,5,7,19], k = 2

Output

```
>>> ===== RESTART =====  
>>>  
5
```

Result

The program finds the k-th smallest element in the array.
Time complexity is $O(n \log n)$.

12. To Implement a function `median_of_medians(arr, k)` that takes an unsorted array `arr` and an integer `k`, and returns the k-th smallest element in the array.

arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] k = 6

arr = [23, 17, 31, 44, 55, 21, 20, 18, 19, 27] k = 5

Output: An integer representing the k-th smallest element in the array.

Aim

To implement a function that returns the k-th smallest element using Median of Medians technique.

Algorithm

1. Divide array into small groups.
2. Compute medians of groups.
3. Use median of medians as pivot.
4. Partition array recursively.
5. Return k-th smallest element.

Python Code

```
File Edit Format Run Options Windows Help
def median_of_medians(arr,k):

    if len(arr)<=5:
        return sorted(arr)[k-1]

    groups=[arr[i:i+5] for i in range(0,len(arr),5)]

    medians=[sorted(g)[len(g)//2] for g in groups]

    pivot=median_of_medians(medians,len(medians)//2+1)

    left=[x for x in arr if x<pivot]
    right=[x for x in arr if x>pivot]
    mid=[x for x in arr if x==pivot]

    if k<=len(left):
        return median_of_medians(left,k)

    elif k<=len(left)+len(mid):
        return pivot

    else:
        return median_of_medians(right,k-len(left)-len(mid))

print(median_of_medians([1,2,3,4,5,6,7,8,9,10],6))

print(median_of_medians([23,17,31,44,55,21,20,18,19,27],5))
|
```

Input

arr = [1,2,3,4,5,6,7,8,9,10], k = 6

arr = [23,17,31,44,55,21,20,18,19,27], k = 5

Output

```
>>> ===== RESTART =====
>>>
6
21
>>> |
```

Result

The program efficiently finds the required element using Divide and Conquer. Worst case time complexity is $O(n)$.

13. Write a program to implement Meet in the Middle Technique. Given an array of integers and a target sum, find the subset whose sum is closest to the target. You will use the Meet in the Middle technique to efficiently find this subset.

a) Set[] = {45, 34, 4, 12, 5, 2} Target Sum : 42

b) Set[] = {1, 3, 2, 7, 4, 6} Target sum = 10:

Aim

To find the subset whose sum is closest to the target sum using Meet in the Middle technique.

Algorithm

1. Divide array into two halves.
2. Compute medians of groups.
3. Use median of medians as pivot.
4. Partition array recursively.
5. Return k-th smallest element.

Python Code

```
File Edit Format Run Options Windows Help
from itertools import combinations

a=[45,34,4,12,5,2]

target=42

mid=len(a)//2

left=a[:mid]
right=a[mid:]

ls=[]
rs=[]

for i in range(len(left)+1):
    for s in combinations(left,i):
        ls.append(sum(s))

for i in range(len(right)+1):
    for s in combinations(right,i):
        rs.append(sum(s))

best=0

for x in ls:
    for y in rs:
        if abs(target-(x+y))<abs(target-best):
            best=x+y

print(best)
```

Input

Set = [45,34,4,12,5,2]

Target = 42

Output

```
>>> ===== RESTART =====  
>>>  
41  
>>> |
```

Result

The program reduces complexity compared to brute force search.

Time complexity is approximately $O(2^{(n/2)})$.

14. Write a program to implement Meet in the Middle Technique. Given a large array of integers and an exact sum E, determine if there is any subset that sums exactly to E. Utilize the Meet in the Middle technique to handle the potentially large size of the array. Return true if there is a subset that sums exactly to E, otherwise return false.

a) $E = \{1, 3, 9, 2, 7, 12\}$ exact Sum = 15

b) $E = \{3, 34, 4, 12, 5, 2\}$ exact Sum = 15

Aim

To check whether any subset of the array gives the exact target sum.

Algorithm

1. Generate all subset sums.
2. Compare subset sums with target value.
3. If exact sum found $\rightarrow$ return True.
4. Otherwise return False.

Python Code

```
from itertools import combinations

a=[3,34,4,12,5,2]

target=15

mid=len(a)//2

left=a[:mid]
right=a[mid:]

ls=[]
rs=[]

for i in range(len(left)+1):
    for s in combinations(left,i):
        ls.append(sum(s))

for i in range(len(right)+1):
    for s in combinations(right,i):
        rs.append(sum(s))

found=False

for x in ls:
    for y in rs:
        if x+y==target:
            found=True

print(found)
```

Input

$E = [1, 3, 9, 2, 7, 12]$

Exact Sum = 15

Output

```
>>> ===== RESTART =====  
>>>  
True  
>>> |
```

Result

The program efficiently checks subset sum existence.

Time complexity is $O(2^{(n/2)})$.

15. Given two 2×2 Matrices A and B

$A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix}$

Use Strassen's matrix multiplication algorithm to compute the product matrix C such that $C = A \times B$.

Test Cases:

Consider the following matrices for testing your implementation: Test

Case 1:

$A = \begin{pmatrix} 1 & 7 \\ 3 & 5 \end{pmatrix}$ $B = \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}$

Expected Output:

$C = \begin{pmatrix} 18 & 14 \\ 62 & 66 \end{pmatrix}$

Aim

To multiply two 2×2 matrices using Strassen's Algorithm.

Algorithm

1. Divide matrices into smaller parts.
2. Compute seven intermediate products.
3. Combine products to form result matrix.
4. Return multiplied matrix.

Python Code

```
File Edit Format Run Options Windows Help
A=[[1,7],[3,5]]
B=[[6,8],[4,2]]

a,b,c,d=A[0][0],A[0][1],A[1][0],A[1][1]
e,f,g,h=B[0][0],B[0][1],B[1][0],B[1][1]

m1=(a+d)*(e+h)
m2=(c+d)*e
m3=a*(f-h)
m4=d*(g-e)
m5=(a+b)*h
m6=(c-a)*(e+f)
m7=(b-d)*(g+h)

c11=m1+m4-m5+m7
c12=m3+m5
c21=m2+m4
c22=m1-m2+m3+m6

print([[c11,c12],[c21,c22]])
```

Input

```
A = [[1,7],[3,5]]
B = [[6,8],[4,2]]
```

Output

```
>>> ===== RESTART =====
>>>
[[34, 22], [38, 34]]
>>> |
```

Result

The program multiplies matrices faster than normal multiplication for large matrices.
Time complexity is $O(n^{2.81})$.

16 Given two integers X=1234 and Y=5678: Use the Karatsuba algorithm to compute the product $Z=X \times Y$

Test Case 1:

Input: $x=1234, y=5678$

Expected Output: $z=1234 \times 5678 = 7016652$

Aim

To multiply two large integers using Karatsuba Algorithm.

Algorithm

1. Split numbers into two halves.
2. Compute three recursive multiplications.
3. Combine results using Karatsuba formula.
4. Return final product.

Python Code

```
File Edit Format Run Options Windows Help
def karatsuba(x,y):

    if x<10 or y<10:
        return x*y

    n=max(len(str(x)),len(str(y)))
    m=n//2

    a=x//10**m
    b=x%10**m

    c=y//10**m
    d=y%10**m

    ac=karatsuba(a,c)
    bd=karatsuba(b,d)
    adbc=karatsuba(a+b,c+d)-ac-bd

    return ac*10**(2*m)+adbc*10**m+bd

print(karatsuba(1234,5678))
```

Input

x = 1234
y = 5678

Output

```
>>> ===== RESTART =====
>>>
7006652
>>>
```

Result

The program multiplies two integers using Karatsuba Algorithm efficiently.
Time complexity is $O(n^{1.58})$.